



VIVEKANANDA GLOBAL UNIVERSITY, JAIPUR



(Established by Act11/2012ofRajasthanGovt.Coveredu/s2(f)ofUGCAct,1956)

Lecture Notes

Course Code - UGCSE201 Course Name - Data Structures

Module 2: Stacks, Queues and Lists

Contents

- Module 2. 1. Definition, Array based implementation of stacks
- Module 2. 2. Examples- Infix, postfix, prefix representation
- Module 2. 3. Applications- Mathematical expression Evaluation
- Module 2. 4. **Queues-Array based implementation of Queues**
- Module 2. 5. Circular implementation of Queues, Double ended queues.
- Module 2. 6. Linked List- Singly linked Lists and Operations associated with it
- Module 2. 7. Stack and Queue Implementation Using linked list,
- Module 2. 8. Straight circular implementation of doubly linked Lists, Priority queues

2.4.1 Introduction to Queue

A queue is a sequential storage structure that permits access only at the two ends of the sequence. We refer to the ends of the sequence as the front and rear. A queue inserts new elements at the rear and removes elements from the front of the sequence. You will note that a queue removes elements in the same order in which they were stored, and hence a queue provides FIFO (first-in / first-out), or FCFS (first-come / first-served), ordering.

2.4.2 Operations on Queue:

Mainly the following four basic operations are performed on queue: enqueue(value) - Inserting value into the queue. In a queue data structure, enqueue() is a function used to insert a new element into the queue. In a queue,

the new element is always inserted at rear position. The enqueue() function takes one integer value as parameter and inserts that value into the queue. We can use the following steps to insert an element into the queue...

Step 1: Check whether queue is FULL. ($\text{rear} == \text{SIZE}-1$)

Step 2: If it is FULL, then display "Queue is FULL!!! Insertion is not possible!!!" and terminate the function.

Step 3: If it is NOT FULL, then increment rear value by one (rear++) and set queue[rear] = value.

dequeue() - Deleting a value from the Queue

In a queue data structure, dequeue() is a function used to delete an element from the queue. In a queue, the element is always deleted from front position. The dequeue() function does not take any value as parameter. We can use the following steps to delete an element from the queue...

Step 1: Check whether queue is EMPTY. (front == rear)

Step 2: If it is EMPTY, then display "Queue is EMPTY!!! Deletion is not possible!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then increment the front value by one (front ++). Then display queue[front] as deleted element. Then check whether both front and rear are equal (front == rear), if it TRUE, then set both front and rear to '-1' (front = rear = -1).

display() - Displays the elements of a Queue

We can use the following steps to display the elements of a queue...

Step 1: Check whether queue is EMPTY. (front == rear)

Step 2: If it is EMPTY, then display "Queue is EMPTY!!!" and terminate the function.

Step 3: If it is NOT EMPTY, then define an integer variable 'i' and set 'i = front+1'.

Step 4: Display 'queue[i]' value and increment 'i' value by one (i++). Repeat the same until 'i' value is equal to rear (i <= rear)

Front: Get the front item from queue. Rear: Get the last item from queue.

Graphical Presentation:

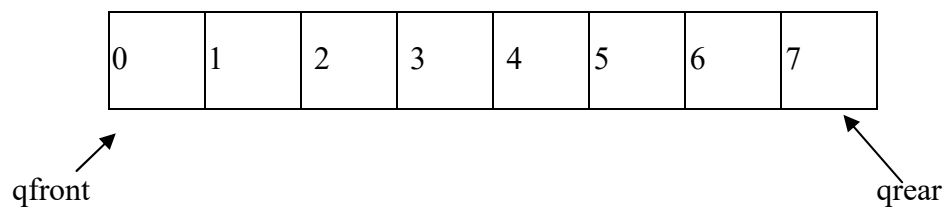


Figure 2.1: Queue

Time Complexity:

Time complexity of all operations like enqueue(), dequeue(), isFull(), isEmpty(), front() and rear() is $O(1)$. There is no loop in any of the operations

QUEUES AS ADT:

In case of Queue we know that what to implement but how to implement is not known, hence queue is called ADT.

DIFFERENT IMPLEMENTATION OF QUEUE:

- Circular Queue.
- Priority Queue.
- Deque